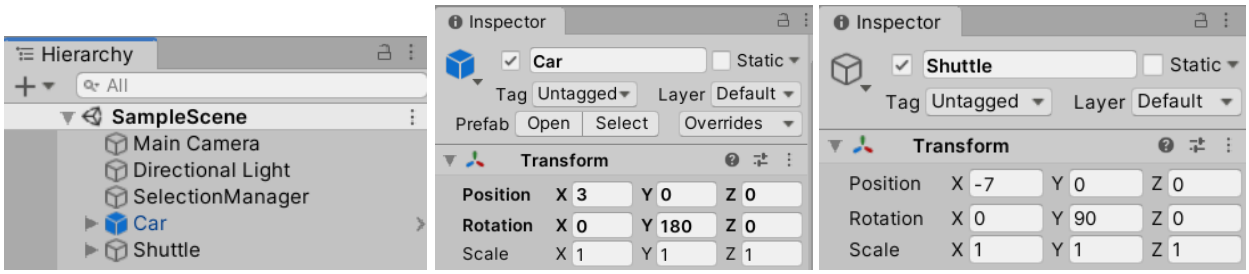
Speech Input

For Windows 10 Unity has a built-in API for speech recognition in the Unity.Windows.Speech namespace. You can use the KeywordRecognizer, GrammarRecognizer and DictationRecognizer classes. In this lab we will use the KeywordRecognizer class - https://docs.unity3d.com/ScriptReference/Windows.Speech.KeywordRecognizer.html.

The task is to allow the user to select different objects and give commands to the objects using voice commands.

Create a new 3D project in Unity. Import several 3D models and an audio asset – the examples here will use the transport shuttle and car 3D models and the AccelerationHigh.wav files.

Add the shuttle and car to the scene (rename the game objects in the Hierarchy to *Car* and *Shuttle*). Place them at different positions – we will later move the camera depending on the current selection. Add an empty game object (right click in the Hierarchy → Create Empty) and name it SelectionManager.



Create a new C# script named SpeechInput.cs (right click in the Project window → Create →C# Script) and open it. At the top of the script add three new namespaces:

```
using UnityEngine.Events;
using UnityEngine.Windows.Speech; //contains the classes for speech input
using System.Linq;//contains classes that simplify working with lists, dictionaries, etc.
```

Embedded in the SpeechInput class add a new public class called CommandHandler with two public fields – *string phrase* and *UnityEvent handler* – this class defines a pair of a phrase and a handler to be executed when the phrase is recognized. This class will be used to add pairs of words and handlers in the Unity editor in design time. To allow the Unity editor to work with instances of the class the class must be serializable – add a [System.Serializable()] attribute on the line above the CommandHandler class name. The class looks like this:

```
[System.Serializable()]
public class CommandHandler
{
    public string phrase;
    public UnityEvent handler;

}
```

Add a public list of the *CommandHandler* type called *commands* to the SpeechInput class. Also add a private *KeywordRecognizer recognizer* field. In the Awake/Start method initialize the recognizer instance with the words that are to be recognized and add a handler to the OnPhraseRecognized event. To get the words we will use the Linq Select method to create an array with all words from the *commands* list:

```
private void Awake()
{
    recognizer = new KeywordRecognizer(commands.Select(x => x.phrase).ToArray());
    recognizer.OnPhraseRecognized += Recognizer_OnPhraseRecognized;
}
```

We will enable and disable the KeywordRecognizer when the object is enabled / disabled – add the OnEnable and OnDisable Unity methods to the SpeechInput class and in them call the Start/ Stop methods of the KeywordRecognizer:

```
    private void OnEnable()
    {
        recognizer.Start();
    }

    private void OnDisable()
    {
        recognizer.Stop();

    }
```

In the event handler for the OnPhraseRecognized event we will find the recognized command in the *commands* list and call its handler:

```
    private void Recognizer_OnPhraseRecognized(PhraseRecognizedEventArgs args)
    {
        CommandHandler ch = commands.First(x => x.phrase == args.text);
        if(ch != null && ch.handler != null)
        {
            ch.handler.Invoke();
        }
    }
```

The args parameter contains information about the recognized phrase. Using the Linq First method we search for an element in the *commands* list whose word field equals the recognized text (in args.text). If such an element is found and it has registered handlers, they are invoked.

We will add another method called ChangeHandler that will allow us to change the handler for a specific command. The method accepts two parameters – the phrase for which we change the handler and a UnityAction pointing to the new handler method. It will then remove all current handlers and add the new one using the UnityEvent's RemoveAllListeners and AddListener methods.

```
    public void ChangeHandler(string phrase, UnityAction newHandler)
    {
        UnityEvent ue = commands.First(x => x.phrase == phrase).handler;
        ue.RemoveAllListeners();
        ue.AddListener(newHandler);
    }
```

The whole SpeechInput class looks like this:

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;
using UnityEngine.Windows.Speech;
using System.Linq;

public class SpeechInput : MonoBehaviour
{
    [System.Serializable()]
    public class CommandHandler
    {
        public string phrase;
        public UnityEvent handler;
    }

    public List<CommandHandler> commands;
    private KeywordRecognizer recognizer;

    private void Awake()
    {
        recognizer = new KeywordRecognizer(commands.Select(x => x.phrase).ToArray());
        recognizer.OnPhraseRecognized += Recognizer_OnPhraseRecognized;
    }
```

```
    private void Recognizer_OnPhraseRecognized(PhraseRecognizedEventArgs args)
    {
        CommandHandler ch = commands.First(x => x.phrase == args.text);
        if(ch != null && ch.handler != null)
        {
            ch.handler.Invoke();
        }
    }

    private void OnEnable()
    {
        recognizer.Start();
    }

    private void OnDisable()
    {
        recognizer.Stop();
    }

    public void ChangeHandler(string phrase, UnityAction newHandler)
    {
        UnityEvent ue = commands.First(x => x.phrase == phrase).handler;
        ue.RemoveAllListeners();
        ue.AddListener(newHandler);
    }
}
```

In Unity add the SpeechInput script to the SelectionManager game object. You will see the Commands list in the Inspector, but we will adjust the commands themselves later.

Create a new C# script named Selectable.cs. It will be attached to objects that can be selected (in this case the car and the shuttle) and will be used to define additional properties for the object. The class will actually declare only a public Vector3 viewPos; that will be used to specify the camera position when this is the currently selected object:

```
using UnityEngine;
public class Selectable : MonoBehaviour
{
    public Vector3 viewPos;
}
```

In Unity add the Selectable script to the Car and Shuttle game objects. Set the values for the viewPos for the two objects. For example (Car – left, Shuttle – right):

Create a new C# script named Playable.cs. It will be attached to the objects that can be interacted with. In this class we will add public void Play and public void Stop methods that will be used to make the object do something or stop doing it. Declare two public UnityEvent fields – OnPlay and OnStop that will allow to add handlers to handle the Play and Stop actions and in the Play and Stop methods invoke the event handlers if the event fields are not null. The Playable script:

```
using UnityEngine;
using UnityEngine.Events;

public class Playable : MonoBehaviour
{
    public UnityEvent OnPlay;
    public UnityEvent OnStop;

    public void Play()
```

```
    {
        if(OnPlay != null)
        {
            OnPlay.Invoke();
        }
    }

    public void Stop()
    {
        if(OnStop != null)
        {
            OnStop.Invoke();
        }
    }
}
```

In Unity attach the Playable script to the Car and the Shuttle. For the Shuttle create an Animation (as in the ARShuttle lab). Then for the OnPlay event add an event handler set the game object to the Shuttle and from the Function combo box choose Animator→SetTrigger and pass as parameter the name of the trigger starting the animation. In the example the name of the trigger is *play*.

For the Car – drag the audio file on the Car game object or its Inspector to add an AudioSource component. Uncheck the Play on awake combo box and check the Loop check box. Then add an event handler for the OnPlay event, set the game object to the Car and from the Function combo box choose AudioSource→Play. Add an event handler for the OnStop event, set the game object to the Car and from the Function combo box choose AudioSource→Stop.

Create a new C# script named SelectionManager.cs. Open the script and add a public property of type Selectable called currentSelection – it will store the currently selected object. We will also want to be able to control the selected object through the selection manager using speech, so add another private field of type SpeechInput called *speechInput*. We will use the speechInput field to redirect the same voice commands to the currently selected object. The *speechInput* instance is initialized in the Awake/Start method using the GameObject.FindObjectOfType method. Add a public void Select(string objName) method to the SelectionManager class. We will use it to select an object by name using the GameObject.Find method. The SelectionManager class:

```
using UnityEngine;

public class SelectionManager : MonoBehaviour
```

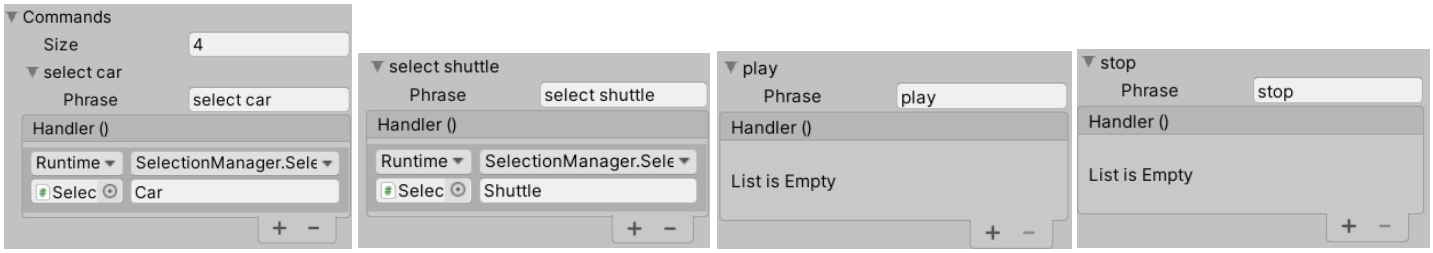
```
{
    public Selectable currentSelection { get; set; }
    private SpeechInput speechInput;

    void Awake()
    {
        speechInput = FindObjectOfType<SpeechInput>();
    }

    public void Select(string objName)
    {
        GameObject obj;
        if((obj = GameObject.Find(objName)) != null)
        {
            currentSelection = obj.GetComponent<Selectable>();
            if (currentSelection == null)
            {
                Debug.Log(objName + " is not selectable");
            }
            else
            {
                //move camera to position to view the object
                Camera.main.transform.position = currentSelection.viewPos;
                //direct camera to look at the object
                Camera.main.transform.LookAt(currentSelection.transform.position);
                //change the handlers for the play and stop commands
                speechInput.ChangeHandler("play", currentSelection.GetComponent<Playable>().Play);
                speechInput.ChangeHandler("stop", currentSelection.GetComponent<Playable>().Stop);
                Debug.Log("selected " + objName);
            }
        }
        else
        {
            Debug.Log("Object " + objName + " not found");
        }
    }
}
```

In Unity add the SelectionManager script to the SelectionManager game object.

In the SpeechInput component of the SelectinManager game object set the Size of the Commands list to 4. Expand each element. For the first element add *select car* for the Phrase, press the plus sign for the Handler, set the game object to the SelectionManager, from the Function combo box choose SelectionManager→Select and in the parameter field enter *Shuttle*. It is important the text in the parameter field matches exactly the name of the game object, including case.

Do the same for the second element, but this time set Phrase to *select shuttle*, and in the parameter field enter *Car*.

Set the other two phrases to *play* and *stop* but leave the handlers empty. They will be set when an object is selected by the ChangeHandler method of the SelectionManager.